

UAS Pengolahan Citra Digital — Pipeline Image Classification

Notebook ini dibuat untuk memenuhi struktur UAS: **Read Dataset → EDA → Data Preprocessing → Split Data → Training → Evaluation → Predict.**

Catatan penggunaan: dataset harus berbentuk folder per kelas, contoh:

```
dataset/  
├── banana/  
│   ├── img1.jpg  
│   └── img2.jpg  
├── apple/  
│   ├── img1.jpg  
│   └── img2.jpg  
└── orange/  
    ├── img1.jpg  
    └── img2.jpg
```

Jalankan notebook ini di Google Colab dari atas ke bawah.

1. Setup Library

```
# Mengimpor library os untuk mengakses path folder dan file.  
import os # Library untuk operasi sistem file.  
  
# Mengimpor library random untuk mengambil contoh gambar secara acak.  
import random # Library untuk proses randomisasi data.  
  
# Mengimpor library shutil untuk menyalin atau mengelola file/folder jika dibutuhkan.  
import shutil # Library untuk operasi copy, move, dan delete file.  
  
# Mengimpor library pathlib agar pengelolaan path file lebih rapi.  
from pathlib import Path # Class Path untuk mengelola lokasi file dan folder.  
  
# Mengimpor library numpy untuk operasi numerik array.  
import numpy as np # Library utama untuk komputasi numerik.  
  
# Mengimpor library pandas untuk membuat tabel ringkasan data.  
import pandas as pd # Library untuk manipulasi data tabular.  
  
# Mengimpor matplotlib untuk visualisasi grafik dan gambar.  
import matplotlib.pyplot as plt # Library visualisasi dasar.  
  
# Mengimpor Image dari PIL untuk membaca dan menampilkan citra.  
from PIL import Image # Library untuk membuka dan memproses file gambar.  
  
# Mengimpor TensorFlow sebagai framework deep learning.  
import tensorflow as tf # Framework machine learning/deep learning.  
  
# Mengimpor ImageDataGenerator untuk augmentasi dan split dataset.  
from tensorflow.keras.preprocessing.image import ImageDataGenerator # Generator citra untuk training dan validasi.  
  
# Mengimpor MobileNetV2 sebagai model transfer learning yang ringan.  
from tensorflow.keras.applications import MobileNetV2 # Arsitektur CNN ringan untuk klasifikasi citra.  
  
# Mengimpor preprocess_input untuk menyesuaikan input dengan MobileNetV2.  
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input # Fungsi preprocessing khusus MobileNetV2.  
  
# Mengimpor layer-layer Keras untuk membangun model klasifikasi.  
from tensorflow.keras import layers, models # Modul layer dan model Keras.  
  
# Mengimpor callback untuk menyimpan model terbaik dan menghentikan training otomatis.  
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau # Callback untuk optimasi training.  
  
# Mengimpor confusion_matrix dan classification_report untuk evaluasi model.  
from sklearn.metrics import confusion_matrix, classification_report # Metrik evaluasi klasifikasi.  
  
# Mengimpor seaborn opsional untuk heatmap confusion matrix.  
import seaborn as sns # Library visualisasi heatmap.  
  
# Mengatur seed numpy agar hasil eksperimen lebih konsisten.  
np.random.seed(42) # Seed untuk random numpy.
```

```
# Mengatur seed TensorFlow agar training lebih reproducible.
tf.random.set_seed(42) # Seed untuk random TensorFlow.

# Mengecek versi TensorFlow yang digunakan di environment Colab.
print("TensorFlow version:", tf.__version__) # Menampilkan versi TensorFlow.
```

TensorFlow version: 2.20.0

2. Read Dataset

```
# Menghubungkan Google Drive ke Google Colab.
from google.colab import drive # Modul Colab untuk mount Google Drive.

# Melakukan mount Google Drive agar dataset bisa dibaca dari Drive.
drive.mount('/content/drive') # Mount Drive ke folder /content/drive.
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
# Menentukan path dataset utama yang berisi folder kelas.
DATASET_DIR = "/content/drive/MyDrive/dataset_pcd" # Ganti path ini sesuai lokasi dataset kalian.

# Mengubah string path dataset menjadi objek Path.
dataset_path = Path(DATASET_DIR) # Membuat objek path untuk dataset.

# Mengecek apakah folder dataset tersedia.
if not dataset_path.exists(): # Kondisi jika folder dataset belum ditemukan.
    raise FileNotFoundError(f"Folder dataset tidak ditemukan: {DATASET_DIR}. Silakan ubah DATASET_DIR sesuai lokasi dataset")

# Membuat daftar ekstensi gambar yang diperbolehkan.
valid_extensions = [".jpg", ".jpeg", ".png", ".bmp", ".webp"] # Format gambar yang umum digunakan.

# Mengambil daftar folder kelas dari folder dataset.
class_dirs = [folder for folder in dataset_path.iterdir() if folder.is_dir()] # Mengambil semua subfolder sebagai kelas.

# Mengurutkan nama kelas agar konsisten.
class_dirs = sorted(class_dirs, key=lambda x: x.name.lower()) # Mengurutkan kelas berdasarkan nama folder.

# Mengecek apakah minimal ada dua kelas untuk klasifikasi.
if len(class_dirs) < 2: # Kondisi jika jumlah kelas kurang dari dua.
    raise ValueError("Dataset minimal harus memiliki 2 folder kelas untuk klasifikasi.") # Menampilkan pesan error yang muncul

# Membuat daftar nama kelas dari nama folder.
class_names = [folder.name for folder in class_dirs] # Mengambil nama folder sebagai label kelas.

# Menampilkan daftar kelas yang ditemukan.
print("Kelas ditemukan:", class_names) # Output nama kelas dataset.

Kelas ditemukan: ['buah_busuk', 'buah_segar']
```

3. Exploratory Data Analysis (EDA)

```
# Membuat list kosong untuk menyimpan informasi jumlah gambar per kelas.
data_summary = [] # List untuk ringkasan dataset.

# Melakukan looping pada setiap folder kelas.
for class_dir in class_dirs: # Iterasi setiap kelas.
    image_files = [file for file in class_dir.rglob("*") if file.suffix.lower() in valid_extensions] # Mengambil semua file gambar
    data_summary.append({"class_name": class_dir.name, "total_images": len(image_files)}) # Menyimpan nama kelas dan jumlah gambar

# Mengubah ringkasan dataset menjadi DataFrame.
df_summary = pd.DataFrame(data_summary) # Membuat tabel ringkasan dataset.

# Menampilkan tabel ringkasan jumlah gambar per kelas.
display(df_summary) # Menampilkan DataFrame di notebook.

# Menampilkan total seluruh gambar di dataset.
print("Total gambar:", df_summary["total_images"].sum()) # Output total data gambar.

# Menampilkan jumlah kelas di dataset.
print("Total kelas:", len(class_names)) # Output total kelas.
```

	class_name	total_images	
0	buah_busuk	30	
1	buah_seggar	241	

Total gambar: 271
Total kelas: 2

Langkah berikutnya: [Buat kode dengan df_summary](#) [New interactive sheet](#)

```
# Membuat ukuran figure untuk grafik distribusi kelas.
plt.figure(figsize=(10, 5)) # Menentukan ukuran grafik.

# Membuat bar chart jumlah gambar per kelas.
plt.bar(df_summary["class_name"], df_summary["total_images"]) # Membuat grafik batang.

# Memberi judul grafik distribusi kelas.
plt.title("Distribusi Jumlah Gambar per Kelas") # Judul grafik.

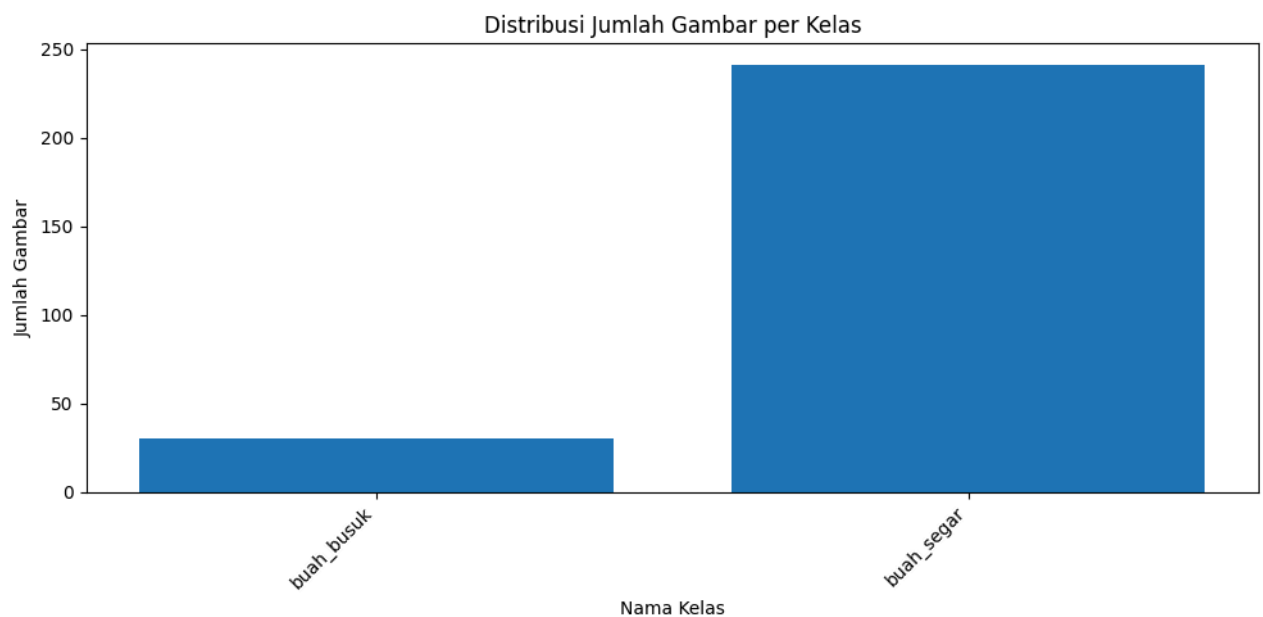
# Memberi label sumbu X.
plt.xlabel("Nama Kelas") # Label sumbu X.

# Memberi label sumbu Y.
plt.ylabel("Jumlah Gambar") # Label sumbu Y.

# Memutar label kelas agar tidak bertumpuk.
plt.xticks(rotation=45, ha="right") # Rotasi label sumbu X.

# Merapikan layout grafik.
plt.tight_layout() # Mengatur layout agar rapi.

# Menampilkan grafik.
plt.show() # Output grafik.
```



```
# Membuat list kosong untuk menyimpan path contoh gambar.
sample_images = [] # List path gambar contoh.

# Mengambil maksimal tiga contoh gambar dari setiap kelas.
for class_dir in class_dirs: # Iterasi setiap folder kelas.
    image_files = [file for file in class_dir.rglob("*") if file.suffix.lower() in valid_extensions] # Mengambil file gambar
    selected_images = random.sample(image_files, min(3, len(image_files))) # Mengambil maksimal tiga gambar acak.
    sample_images.extend([(img_path, class_dir.name) for img_path in selected_images]) # Menyimpan path gambar dan label ke

# Mengatur jumlah kolom visualisasi.
cols = 3 # Jumlah kolom gambar.

# Menghitung jumlah baris visualisasi berdasarkan jumlah sampel.
rows = int(np.ceil(len(sample_images) / cols)) # Jumlah baris gambar.

# Membuat figure untuk menampilkan contoh gambar.
plt.figure(figsize=(12, 4 * rows)) # Ukuran figure dinamis.

# Melakukan looping untuk menampilkan setiap contoh gambar.
```

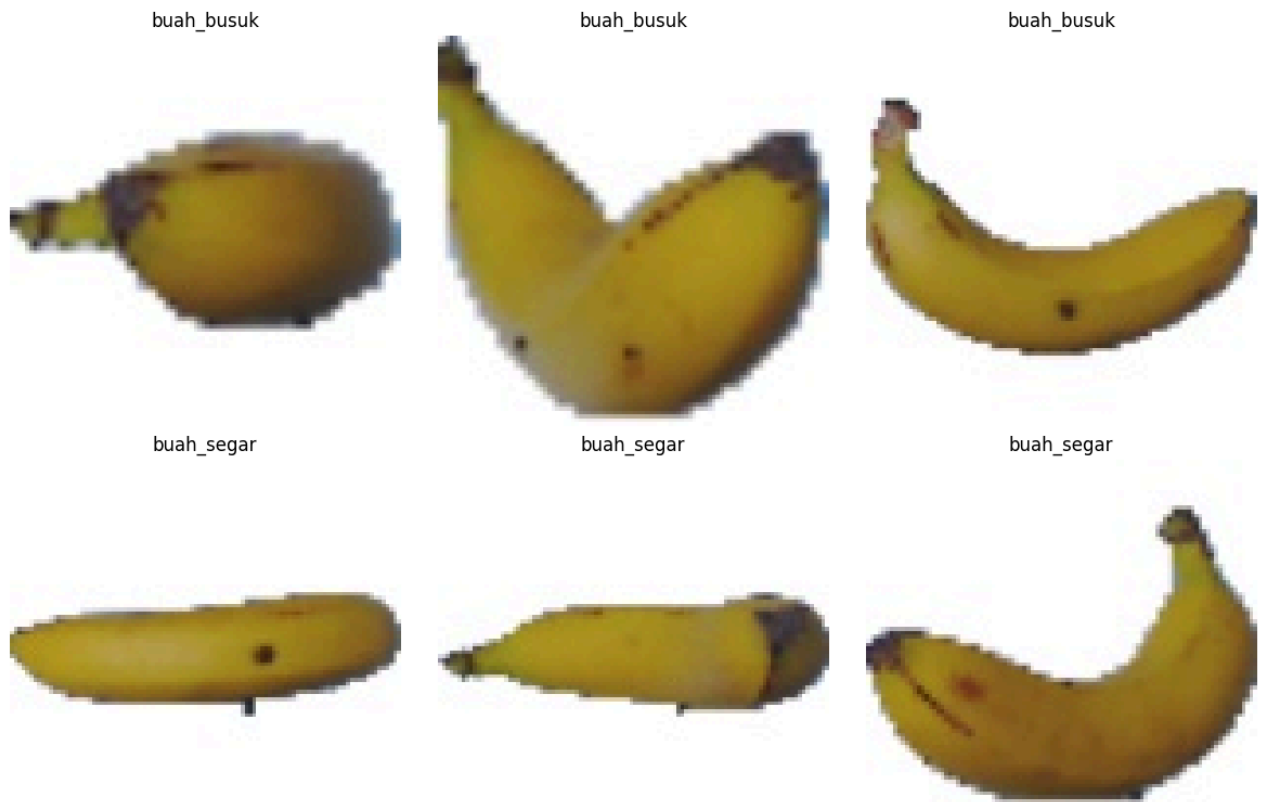
```

for idx, (img_path, label) in enumerate(sample_images): # Iterasi gambar contoh.
    img = Image.open(img_path).convert("RGB") # Membuka gambar dan mengubah ke RGB.
    plt.subplot(rows, cols, idx + 1) # Membuat subplot.
    plt.imshow(img) # Menampilkan gambar.
    plt.title(label) # Menampilkan label kelas.
    plt.axis("off") # Menghilangkan sumbu.

# Merapikan layout gambar.
plt.tight_layout() # Mengatur layout subplot.

# Menampilkan contoh gambar.
plt.show() # Output visualisasi gambar.

```



```

# Membuat list kosong untuk menyimpan ukuran gambar.
image_sizes = [] # List ukuran gambar.

# Mengambil informasi ukuran gambar dari maksimal 500 gambar agar proses tetap ringan.
all_image_files = [file for file in dataset_path.rglob("*") if file.suffix.lower() in valid_extensions] # Mengambil semua

# Mengambil sampel gambar untuk analisis ukuran.
sample_for_size = random.sample(all_image_files, min(500, len(all_image_files))) # Sampel maksimal 500 gambar.

# Melakukan looping untuk membaca ukuran setiap gambar.
for img_path in sample_for_size: # Iterasi setiap gambar sampel.
    img = Image.open(img_path) # Membuka gambar.
    image_sizes.append(img.size) # Menyimpan ukuran width dan height.

# Mengubah list ukuran gambar menjadi DataFrame.
df_sizes = pd.DataFrame(image_sizes, columns=["width", "height"]) # Membuat tabel ukuran gambar.

# Menampilkan statistik ukuran gambar.
display(df_sizes.describe()) # Menampilkan statistik deskriptif ukuran gambar.

# Membuat grafik sebaran ukuran gambar.
plt.figure(figsize=(7, 5)) # Ukuran figure grafik.

# Membuat scatter plot width dan height.
plt.scatter(df_sizes["width"], df_sizes["height"], alpha=0.5) # Visualisasi sebaran ukuran gambar.

# Memberi judul grafik.
plt.title("Sebaran Ukuran Gambar") # Judul grafik.

```

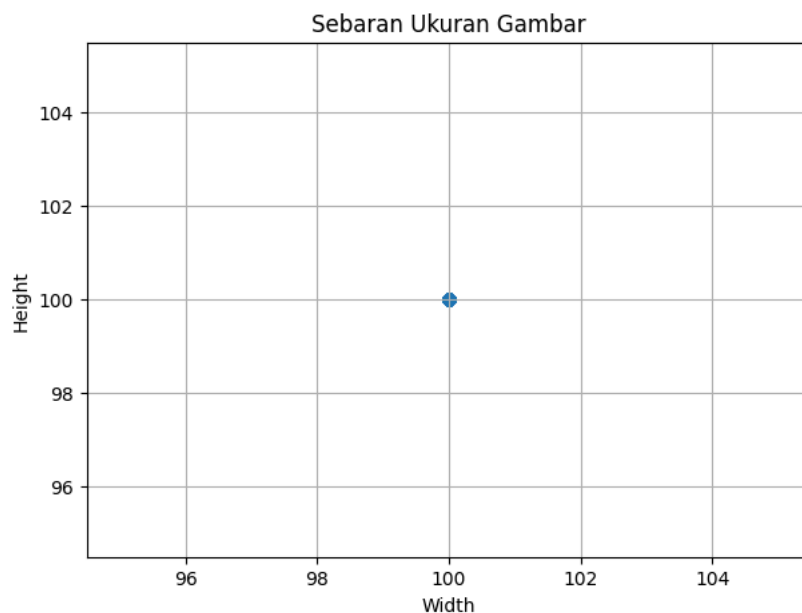
```
# Memberi label sumbu X.
plt.xlabel("Width") # Label sumbu X.

# Memberi label sumbu Y.
plt.ylabel("Height") # Label sumbu Y.

# Menampilkan grid agar mudah dibaca.
plt.grid(True) # Mengaktifkan grid.

# Menampilkan grafik.
plt.show() # Output scatter plot.
```

	width	height	
count	482.0	482.0	
mean	100.0	100.0	
std	0.0	0.0	
min	100.0	100.0	
25%	100.0	100.0	
50%	100.0	100.0	
75%	100.0	100.0	
max	100.0	100.0	



4. Data Preprocessing dan Split Data

```
# Menentukan ukuran input gambar untuk model MobileNetV2.
IMG_SIZE = (224, 224) # Ukuran gambar standar model transfer learning.

# Menentukan jumlah data yang diproses setiap batch.
BATCH_SIZE = 32 # Batch size umum untuk Colab.

# Menentukan rasio data validasi.
VALIDATION_SPLIT = 0.2 # 20% data digunakan untuk validasi.

# Membuat data generator untuk training dengan augmentasi.
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input, # Normalisasi sesuai standar MobileNetV2.
    rotation_range=20, # Augmentasi rotasi gambar maksimal 20 derajat.
    width_shift_range=0.1, # Augmentasi geser horizontal maksimal 10%.
    height_shift_range=0.1, # Augmentasi geser vertikal maksimal 10%.
    zoom_range=0.15, # Augmentasi zoom gambar maksimal 15%.
    horizontal_flip=True, # Augmentasi flip horizontal.
    validation_split=VALIDATION_SPLIT # Split otomatis training dan validation.
) # Generator untuk data training.

# Membuat data generator untuk validasi tanpa augmentasi tambahan.
valid_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input, # Normalisasi sesuai standar MobileNetV2.
    validation_split=VALIDATION_SPLIT # Split validasi sama dengan training generator
```

```

) # Generator untuk data validasi.

# Membuat generator data training dari struktur folder dataset.
train_generator = train_datagen.flow_from_directory(
    DATASET_DIR, # Path dataset.
    target_size=IMG_SIZE, # Resize semua gambar ke ukuran input model.
    batch_size=BATCH_SIZE, # Jumlah gambar per batch.
    class_mode="categorical", # Label multi-class dalam format one-hot.
    subset="training", # Mengambil subset training.
    shuffle=True, # Mengacak data training.
    seed=42 # Seed agar split konsisten.
) # Generator data training.

# Membuat generator data validasi dari struktur folder dataset.
valid_generator = valid_datagen.flow_from_directory(
    DATASET_DIR, # Path dataset.
    target_size=IMG_SIZE, # Resize semua gambar ke ukuran input model.
    batch_size=BATCH_SIZE, # Jumlah gambar per batch.
    class_mode="categorical", # Label multi-class dalam format one-hot.
    subset="validation", # Mengambil subset validasi.
    shuffle=False, # Tidak mengacak validasi agar evaluasi konsisten.
    seed=42 # Seed agar split konsisten.
) # Generator data validasi.

# Mengambil dictionary mapping nama kelas ke index label.
class_indices = train_generator.class_indices # Mapping kelas ke angka.

# Membalik dictionary agar index bisa dikembalikan ke nama kelas.
idx_to_class = {v: k for k, v in class_indices.items()} # Mapping angka ke nama kelas.

# Menampilkan mapping kelas.
print("Mapping kelas:", class_indices) # Output mapping label.

```

```

Found 386 images belonging to 2 classes.
Found 96 images belonging to 2 classes.
Mapping kelas: {'buah_busuk': 0, 'buah_segar': 1}

```

5. Training Model

```

# Menghitung jumlah kelas dari generator training.
num_classes = train_generator.num_classes # Jumlah kelas output model.

# Mencoba memuat MobileNetV2 dengan bobot ImageNet.
try: # Blok percobaan untuk transfer learning.
    base_model = MobileNetV2(weights="imagenet", include_top=False, input_shape=(224, 224, 3)) # Backbone pretrained ImageNet
except Exception as error: # Jika bobot ImageNet gagal diunduh.
    print("Gagal memakai pretrained ImageNet, model akan memakai bobot random:", error) # Pesan fallback.
    base_model = MobileNetV2(weights=None, include_top=False, input_shape=(224, 224, 3)) # Backbone tanpa pretrained weight

# Membekukan layer backbone agar training awal lebih cepat dan stabil.
base_model.trainable = False # Layer MobileNetV2 tidak dilatih pada tahap awal.

# Membuat input layer sesuai ukuran gambar.
inputs = layers.Input(shape=(224, 224, 3)) # Input model citra RGB.

# Menghubungkan input ke base model.
x = base_model(inputs, training=False) # Ekstraksi fitur dari backbone.

# Mengubah feature map menjadi vektor menggunakan global average pooling.
x = layers.GlobalAveragePooling2D()(x) # Pooling fitur spasial.

# Menambahkan dropout untuk mengurangi overfitting.
x = layers.Dropout(0.3)(x) # Dropout 30%.

# Menambahkan output layer sesuai jumlah kelas.
outputs = layers.Dense(num_classes, activation="softmax")(x) # Output probabilitas kelas.

# Membuat model final dari input dan output.
model = models.Model(inputs, outputs) # Model klasifikasi citra.

# Melakukan compile model dengan optimizer Adam.
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001), # Optimizer Adam dengan learning rate kecil.
    loss="categorical_crossentropy", # Loss untuk klasifikasi multi-class.
    metrics=["accuracy"] # Metrik utama akurasi.
) # Compile model.

# Menampilkan ringkasan arsitektur model.
model.summary() # Output struktur model.

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_9406464/9406464 2s 0us/step

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 2)	2,562

Total params: 2,260,546 (8.62 MB)

Trainable params: 2,562 (10.01 KB)

Non-trainable params: 2,257,984 (8.61 MB)

```
# Menentukan nama file untuk menyimpan model terbaik.
BEST_MODEL_PATH = "best_model_pcd.h5" # File output model terbaik.

# Membuat callback early stopping agar training berhenti saat validasi tidak membaik.
early_stop = EarlyStopping(
    monitor="val_loss", # Metrik yang dipantau.
    patience=5, # Jumlah epoch toleransi tanpa perbaikan.
    restore_best_weights=True # Mengembalikan bobot terbaik.
) # Callback early stopping.

# Membuat callback checkpoint untuk menyimpan model dengan validasi terbaik.
checkpoint = ModelCheckpoint(
    BEST_MODEL_PATH, # Lokasi file model.
    monitor="val_accuracy", # Metrik yang dipantau.
    save_best_only=True, # Hanya menyimpan model terbaik.
    mode="max" # Karena accuracy ingin dimaksimalkan.
) # Callback model checkpoint.

# Membuat callback untuk menurunkan learning rate jika performa stagnan.
reduce_lr = ReduceLROnPlateau(
    monitor="val_loss", # Metrik yang dipantau.
    factor=0.2, # Faktor penurunan learning rate.
    patience=3, # Toleransi epoch sebelum learning rate diturunkan.
    min_lr=1e-7 # Learning rate minimum.
) # Callback pengatur learning rate.

# Menentukan jumlah epoch training awal.
EPOCHS = 15 # Jumlah epoch training.

# Melatih model menggunakan data training dan validasi.
history = model.fit(
    train_generator, # Data training.
    validation_data=valid_generator, # Data validasi.
    epochs=EPOCHS, # Jumlah epoch.
    callbacks=[early_stop, checkpoint, reduce_lr] # Callback training.
) # Proses training model.
```

```
Epoch 1/15
13/13 ----- 0s 1s/step - accuracy: 0.4870 - loss: 0.8610WARNING:absl:You are saving your model as an HDF5 file
13/13 ----- 43s 2s/step - accuracy: 0.4922 - loss: 0.8725 - val_accuracy: 0.5312 - val_loss: 0.7558 - learnir
Epoch 2/15
13/13 ----- 0s 412ms/step - accuracy: 0.4319 - loss: 1.0071WARNING:absl:You are saving your model as an HDF5
13/13 ----- 6s 465ms/step - accuracy: 0.4715 - loss: 0.8995 - val_accuracy: 0.5625 - val_loss: 0.7689 - learr
Epoch 3/15
13/13 ----- 7s 557ms/step - accuracy: 0.5363 - loss: 0.8028 - val_accuracy: 0.5312 - val_loss: 0.7434 - learr
Epoch 4/15
13/13 ----- 5s 410ms/step - accuracy: 0.5000 - loss: 0.8353 - val_accuracy: 0.5312 - val_loss: 0.7330 - learr
Epoch 5/15
13/13 ----- 7s 509ms/step - accuracy: 0.5181 - loss: 0.8208 - val_accuracy: 0.5417 - val_loss: 0.7292 - learr
Epoch 6/15
13/13 ----- 6s 431ms/step - accuracy: 0.4456 - loss: 0.9406 - val_accuracy: 0.5521 - val_loss: 0.7246 - learr
Epoch 7/15
13/13 ----- 0s 408ms/step - accuracy: 0.4869 - loss: 0.8545WARNING:absl:You are saving your model as an HDF5
13/13 ----- 6s 492ms/step - accuracy: 0.5181 - loss: 0.8164 - val_accuracy: 0.5833 - val_loss: 0.7283 - learr
Epoch 8/15
13/13 ----- 7s 529ms/step - accuracy: 0.5026 - loss: 0.8035 - val_accuracy: 0.5417 - val_loss: 0.7206 - learr
Epoch 9/15
13/13 ----- 6s 456ms/step - accuracy: 0.5233 - loss: 0.7940 - val_accuracy: 0.5417 - val_loss: 0.7201 - learr
Epoch 10/15
13/13 ----- 7s 538ms/step - accuracy: 0.4715 - loss: 0.8431 - val_accuracy: 0.5625 - val_loss: 0.7321 - learr
Epoch 11/15
```

```

13/13 ————— 5s 410ms/step - accuracy: 0.5078 - loss: 0.8125 - val_accuracy: 0.5833 - val_loss: 0.7222 - learn
Epoch 12/15
13/13 ————— 7s 562ms/step - accuracy: 0.5155 - loss: 0.8097 - val_accuracy: 0.5729 - val_loss: 0.7191 - learn
Epoch 13/15
13/13 ————— 6s 420ms/step - accuracy: 0.4793 - loss: 0.8377 - val_accuracy: 0.5729 - val_loss: 0.7181 - learn
Epoch 14/15
13/13 ————— 7s 518ms/step - accuracy: 0.5544 - loss: 0.7668 - val_accuracy: 0.5729 - val_loss: 0.7178 - learn
Epoch 15/15
13/13 ————— 6s 421ms/step - accuracy: 0.4948 - loss: 0.8265 - val_accuracy: 0.5833 - val_loss: 0.7168 - learn

```

6. Evaluation

```

# Membuat grafik training accuracy dan validation accuracy.
plt.figure(figsize=(8, 5)) # Ukuran grafik.

# Mengambil nilai accuracy dari history training.
plt.plot(history.history["accuracy"], label="Training Accuracy") # Plot training accuracy.

# Mengambil nilai validation accuracy dari history training.
plt.plot(history.history["val_accuracy"], label="Validation Accuracy") # Plot validation accuracy.

# Memberi judul grafik accuracy.
plt.title("Grafik Accuracy") # Judul grafik.

# Memberi label sumbu X.
plt.xlabel("Epoch") # Label epoch.

# Memberi label sumbu Y.
plt.ylabel("Accuracy") # Label accuracy.

# Menampilkan legenda grafik.
plt.legend() # Menampilkan keterangan garis.

# Menampilkan grid.
plt.grid(True) # Mengaktifkan grid.

# Menampilkan grafik.
plt.show() # Output grafik accuracy.

# Membuat grafik training loss dan validation loss.
plt.figure(figsize=(8, 5)) # Ukuran grafik.

# Mengambil nilai loss dari history training.
plt.plot(history.history["loss"], label="Training Loss") # Plot training loss.

# Mengambil nilai validation loss dari history training.
plt.plot(history.history["val_loss"], label="Validation Loss") # Plot validation loss.

# Memberi judul grafik loss.
plt.title("Grafik Loss") # Judul grafik.

# Memberi label sumbu X.
plt.xlabel("Epoch") # Label epoch.

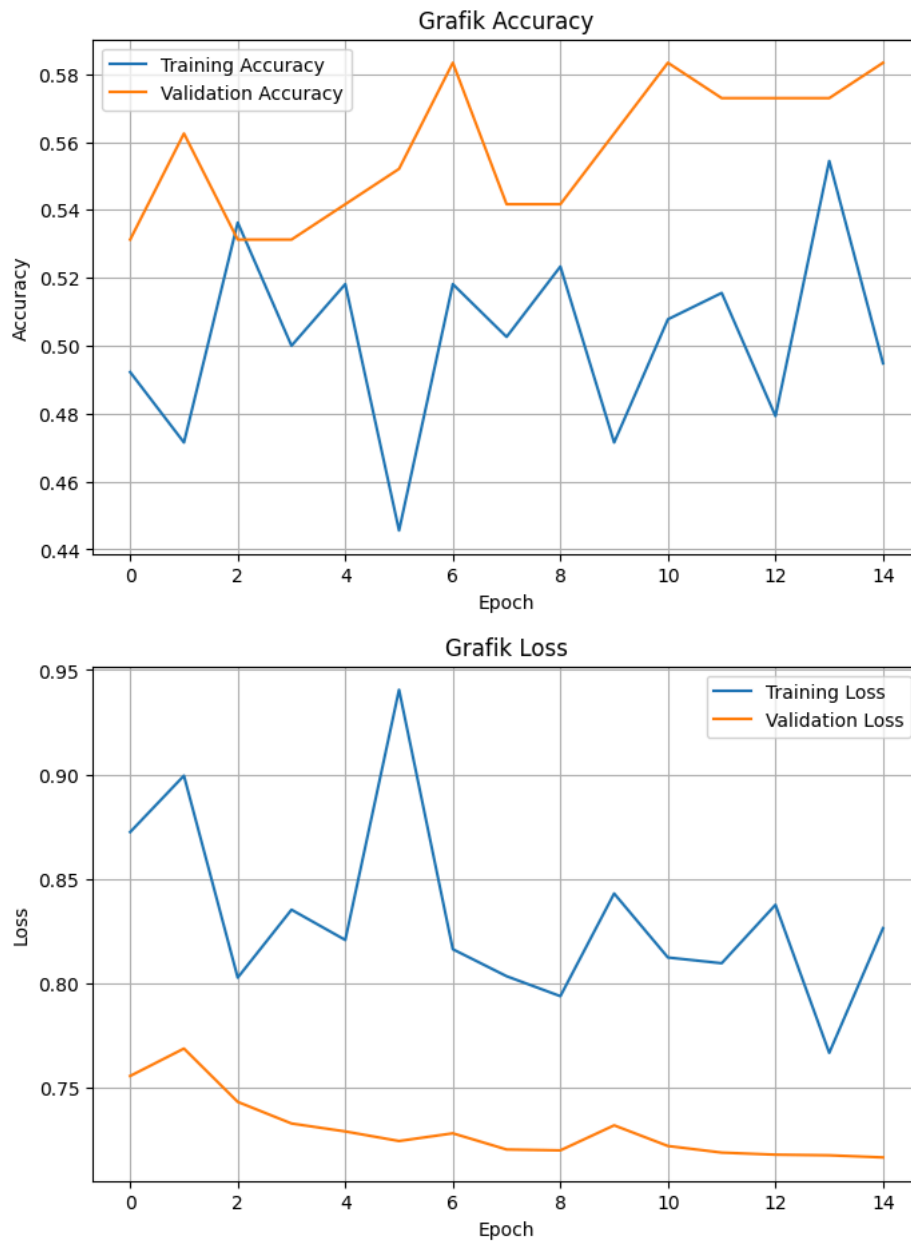
# Memberi label sumbu Y.
plt.ylabel("Loss") # Label loss.

# Menampilkan legenda grafik.
plt.legend() # Menampilkan keterangan garis.

# Menampilkan grid.
plt.grid(True) # Mengaktifkan grid.

# Menampilkan grafik.
plt.show() # Output grafik loss.

```

```
# Mengevaluasi model pada data validasi.
val_loss, val_accuracy = model.evaluate(valid_generator) # Menghitung loss dan accuracy validasi.

# Menampilkan nilai validation loss.
print(f"Validation Loss: {val_loss:.4f}") # Output loss validasi.

# Menampilkan nilai validation accuracy.
print(f"Validation Accuracy: {val_accuracy:.4f}") # Output akurasi validasi.
```

```
# Mengatur ulang generator validasi agar urutan prediksi mulai dari awal.
valid_generator.reset() # Reset pointer data validasi.

# Melakukan prediksi probabilitas pada seluruh data validasi.
y_pred_prob = model.predict(valid_generator) # Output probabilitas tiap kelas.

# Mengambil index kelas dengan probabilitas tertinggi.
y_pred = np.argmax(y_pred_prob, axis=1) # Label prediksi model.

# Mengambil label asli dari generator validasi.
y_true = valid_generator.classes # Label ground truth.

# Membuat confusion matrix.
cm = confusion_matrix(y_true, y_pred) # Matriks perbandingan label asli dan prediksi.

# Membuat figure untuk heatmap confusion matrix.
plt.figure(figsize=(8, 6)) # Ukuran heatmap.

# Menampilkan confusion matrix dalam bentuk heatmap.
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_names, yticklabels=class_names) # Heatmap confusion matrix
```

```
# Memberi judul confusion matrix.
plt.title("Confusion Matrix") # Judul heatmap.

# Memberi label sumbu X sebagai prediksi.
plt.xlabel("Predicted Label") # Label sumbu X.

# Memberi label sumbu Y sebagai label asli.
plt.ylabel("True Label") # Label sumbu Y.

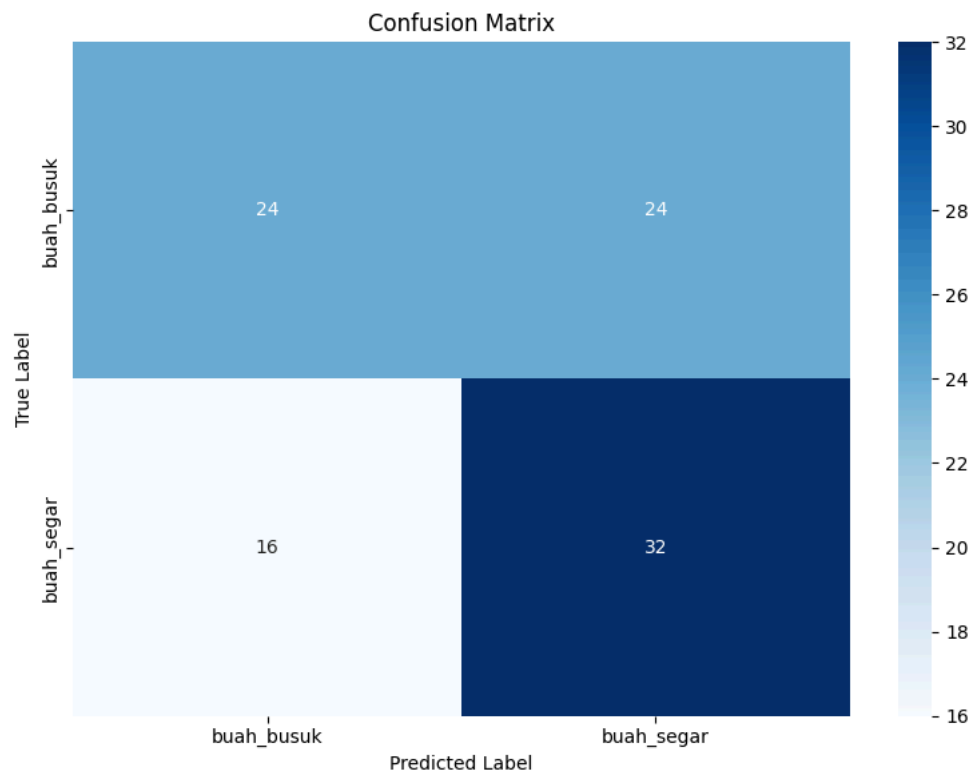
# Merapikan layout visualisasi.
plt.tight_layout() # Layout rapi.

# Menampilkan confusion matrix.
plt.show() # Output heatmap.

# Membuat classification report berisi precision, recall, f1-score, dan support.
report = classification_report(y_true, y_pred, target_names=class_names) # Laporan evaluasi klasifikasi.

# Menampilkan classification report.
print(report) # Output classification report.
```

3/3 ————— 5s 79ms/step



	precision	recall	f1-score	support
buah_busuk	0.60	0.50	0.55	48
buah_segar	0.57	0.67	0.62	48
accuracy			0.58	96
macro avg	0.59	0.58	0.58	96
weighted avg	0.59	0.58	0.58	96

7. Predict Data Baru

```
# Mengimpor modul files dari Google Colab untuk upload gambar uji.
from google.colab import files # Modul upload file di Colab.

# Meminta user mengunggah satu gambar baru untuk diuji.
uploaded = files.upload() # Upload gambar uji dari komputer.

# Mengambil nama file pertama dari hasil upload.
test_image_name = list(uploaded.keys())[0] # Nama file gambar uji.

# Membuka gambar uji dan mengubahnya ke format RGB.
test_img = Image.open(test_image_name).convert("RGB") # Membaca gambar uji.

# Melakukan resize gambar sesuai input model.
test_img_resized = test_img.resize(IMG_SIZE) # Resize ke 224x224.
```

```

# Mengubah gambar menjadi array numpy.
test_array = np.array(test_img_resized) # Konversi gambar ke array.

# Menambahkan dimensi batch karena model menerima input batch.
test_array = np.expand_dims(test_array, axis=0) # Bentuk menjadi 1x224x224x3.

# Melakukan preprocessing sesuai MobileNetV2.
test_array = preprocess_input(test_array) # Normalisasi input.

# Melakukan prediksi probabilitas kelas.
prediction_prob = model.predict(test_array) # Output probabilitas prediksi.

# Mengambil index kelas dengan probabilitas tertinggi.
predicted_index = np.argmax(prediction_prob, axis=1)[0] # Index prediksi tertinggi.

# Mengambil nama kelas berdasarkan index prediksi.
predicted_class = idx_to_class[predicted_index] # Nama kelas hasil prediksi.

# Mengambil confidence score dari probabilitas tertinggi.
confidence = prediction_prob[0][predicted_index] # Nilai confidence prediksi.

# Menampilkan gambar uji.
plt.figure(figsize=(5, 5)) # Ukuran figure gambar.

# Menampilkan gambar asli.
plt.imshow(test_img) # Output gambar uji.

# Menghilangkan sumbu gambar.
plt.axis("off") # Hide axis.

# Memberi judul hasil prediksi.
plt.title(f"Prediksi: {predicted_class} ({confidence:.2%})") # Judul hasil prediksi.

# Menampilkan gambar dan hasil prediksi.
plt.show() # Output visualisasi prediksi.

# Menampilkan hasil prediksi dalam teks.
print(f"Hasil prediksi: {predicted_class}") # Output kelas prediksi.

# Menampilkan tingkat confidence prediksi.
print(f"Confidence: {confidence:.2%}") # Output confidence.

```

Choose Files Banana_8.jpg

Banana_8.jpg(image/jpeg) - 3804 bytes, last modified: 6/26/2022 - 100% done

Saving Banana_8.jpg to Banana_8.jpg

1/1  12s 12s/step

Prediksi: buah_busuk (69.04%)



Hasil prediksi: buah_busuk
Confidence: 69.04%

8. Simpan Model

```

# Menyimpan model akhir ke file .keras.
model.save("final_model_pcd.keras") # Menyimpan model final.

# Menampilkan pesan bahwa model berhasil disimpan.

```

```
print("Model berhasil disimpan sebagai final_model_pcd.keras") # Output status simpan model.
```

```
Model berhasil disimpan sebagai final_model_pcd.keras
```

Catatan Presentasi

Isi slide presentasi bisa dibuat dari poin berikut:

1. Deskripsi dataset dan jumlah kelas.
2. Hasil EDA: distribusi data, contoh gambar, ukuran gambar.
3. Tahap preprocessing: resize, normalisasi, augmentasi.
4. Split data: training 80% dan validation 20%.
5. Model: MobileNetV2 transfer learning.
6. Hasil training: grafik accuracy dan loss.
7. Evaluasi: validation accuracy, confusion matrix, classification report.
8. Demo predict gambar baru.